

Основы радиоэлектроники и связи

Лекция 9. Корректирующие коды

Антон Штанюк

КТПП::ИРИТ::НГТУ / 2026-10-26

Корректирующие коды

Код с проверкой на четность

Код Хэмминга

Корректирующие коды

Что такое корректирующие коды

Корректирующий код (помехоустойчивый код) — код, предназначенный для обнаружения и исправления ошибок, возникающих из-за искажения кодовых последовательностей (например, при передачи по каналу связи, повреждения носителя информации и т.п.)

Выделяют:

1. коды, обнаруживающие ошибки (пример: CRC)
2. коды, исправляющие ошибки (пример: ECC)

Классификация

По способу работы с данными коды бывают:

- блоковые - делят информацию на фрагмены определенной длины, обрабатывают блоки независимо
- сверточные - работают с непрерывным потоком данных

Код с проверкой на четность

Код с проверкой на четность

Один из самых старых, простых и используемых кодов для обнаружения ошибок образуется путем добавления к передаваемой двоичной комбинации, состоящей из k информационных символов (битов), одного контрольного символа (0 или 1), так, чтобы общее число единиц в передаваемой комбинации было четным. Такой код называется кодом с проверкой на четность и позволяет обнаруживать одиночные ошибки.
Например, мы можем использовать один дополнительный бит, добавляя его к ASCII- коду

Код с проверкой на четность

В следующей программе читает файл с ASCII кодировкой (8 бит), для каждого символа вычисляется бит четности и вставляется в закодированную строку для передачи в канал связи.

На приемном конце канала считываются 9 бит, вычисляется сумма единиц в первых 8, а затем справнивается со значением контрольного бита. Если есть несовпадение, то необходимо запросить последний символ заново.

Данный код позволяет обнаруживать ошибку в 1 разряде.

Код с проверкой на четность

```python
def check_even(number):  # функция проверки на четность/нечетность
    return 0 if number % 2 == 0 else 1

def odd_bit(binary_str):
    return check_even(binary_str.count("1"))

# Чтение файла в кодировке ASCII
with open("war_peace.txt", "r", encoding="ascii") as f:
    text = f.read()
```

Код с проверкой на четность

```
# общее число бит
len_bits = 0

for char in text:
    # Преобразуем код символа в двоичную строку формата 8 бит
    binary_str = format(ord(char), '08b')
    #print(char, '->', binary_str)
    odd = odd_bit(binary_str) # бит контроля четности
    binary_str += str(odd)
    len_bits += len(binary_str)

print(len_bits)
```

Код Хэмминга

Расстояние Хэмминга

Расстояние Хэмминга — это число позиций, в которых различаются соответствующие символы двух строк или векторов одинаковой длины.

Измеряет количество замен, необходимых для превращения одной последовательности в другую.
Применяется к двоичным кодам (нулям и единицам), а также к строкам любого алфавита равной длины.

Код Хэмминга

Код Хэмминга — это линейный блочный самоконтролирующийся и самокорректирующийся двоичный код, предложенный Ричардом Хэммингом в 1950 году, который позволяет обнаруживать и исправлять ошибки при передаче или хранении данных

Основные характеристики Принцип работы: в исходную информацию добавляются контрольные биты на позиции, равные степеням двойки (1, 2, 4, 8, ...). Классический пример (7, 4): содержит 4 информационных и 3 контрольных бита (всего 7 бит в слове). Возможности: исправляет одиночные битовые ошибки (1 бит) и обнаруживает двойные ошибки (2 бита). Расстояние Хэмминга: равно 3.

Пример работы кода Хэмминга

Пусть требуется передать слово: 1011

1	2	3	4	5	6	7
c_1	c_2	d_1	c_3	d_2	d_3	d_4
?	?	1	?	0	1	1

Каждый контрольный бит проверяет только те позиции, в двоичной записи которых на соответствующем разряде стоит единица:

- c_1 (позиция 1 = 001) проверяет все позиции, у которых первый справа бит равен 1: это позиции 1, 3, 5, 7.
- c_2 (позиция 2 = 010) проверяет все позиции, у которых второй справа бит равен 1: это позиции 2, 3, 6, 7.
- c_4 (позиция 4 = 100) проверяет все позиции, у которых третий справа бит равен 1: это позиции 4, 5, 6, 7.

Код Хэмминга

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	
0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	1	1	1	0	1	
X		X		X		X		X		X		X		X		X		X		X	1
	X	X			X	X			X	X			X	X			X	X			2
			X	X	X	X					X	X	X	X					X	X	4
							X	X	X	X	X	X	X	X							8
															X	X	X	X	X	X	16

Расчет значений

Используем проверку на четность: сумма контролируемых бит вместе с самим контрольным должна быть четной, то есть давать 0 при сложении по модулю 2
c_1: позиции 3,5,7 ($1 \oplus 0 \oplus 1 = 0$)
c_2: позиции 3,6,7 ($1 \oplus 1 \oplus 1 = 1$)
c_4: позиции 5,6,7 ($0 \oplus 1 \oplus 1 = 0$)
Готовое сообщение для отправки: 0110011.

Поиск и исправление ошибки

Пусть во время передачи изменился 6-й бит и слово стало выглядеть так: 0110011
Необходимо заново рассчитать контрольные биты, включив в сумму и значения самих битов.
Проверяем группу c_1 (позиции 1, 3, 5, 7)
$s_1 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$ - все хорошо
Проверяем группу c_2 (позиции 2, 3, 6, 7)
$s_2 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$ - ошибка!
Проверяем группу c_4 (позиции 4, 5, 6, 7)
$s_4 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$ - ошибка!

Определяем позицию ошибки

Если записать результаты проверок (синдромы) в обратном порядке — от старшего к младшему ($s_4 s_2 s_1$), — мы получим двоичный номер поврежденного бита.

$110 \rightarrow 6$

Ошибка находится в 6 позиции

Исправление ошибки

Инвертируем значение 6 разряда
Получаем исходное кодовое слово: 0110011
Убираем из него контрольные биты (на 1, 2 и 4 местах) и забираем чистые данные (места 3, 5, 6, 7): это 1011.

Пример программной реализации

```
def encode_hamming(data):
    # data: список из 4 информационных битов, например [1, 0, 1, 1]
    d1, d2, d3, d4 = data
    # Вычисляем проверочные биты (p1, p2, p4) по чётности
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4
    # Собираем кодовое слово (позиции 1, 2, 3, 4, 5, 6, 7)
    # p1 на поз. 1, p2 на поз. 2, d1 на поз. 3, p4 на поз. 4, d2, d3, d4
    encoded = [p1, p2, d1, p4, d2, d3, d4]
    return encoded
```

Пример программной реализации

```
def decode_hamming(encoded):
    # encoded: список из 7 битов
    p1, p2, d1, p4, d2, d3, d4 = encoded
    # Проверяем контрольные суммы
    c1 = p1 ^ d1 ^ d2 ^ d4
    c2 = p2 ^ d1 ^ d3 ^ d4
    c4 = p4 ^ d2 ^ d3 ^ d4
    # Номер ошибочного бита
    error_pos = c1 * 1 + c2 * 2 + c4 * 4
```

Пример программной реализации

```
# Если есть ошибка (error_pos > 0), инвертируем бит
if error_pos != 0:
    print(f"Обнаружена ошибка в бите с индексом (1-based): {error_pos}")
    idx = error_pos - 1
    encoded[idx] = 1 - encoded[idx]
else:
    print("Ошибок не обнаружено")

# Извлекаем исходные информационные биты
# (позиции 3, 5, 6, 7 -> индексы 2, 4, 5, 6)
decoded_data = [encoded[2], encoded[4], encoded[5], encoded[6]]
return decoded_data
```

Пример программной реализации

```
# Пример использования:
original_data = [1, 0, 1, 1]
print("Исходные данные:", original_data)

# Кодируем
code = encode_hamming(original_data)
print("Закодировано (7,4):", code)

# Имитируем ошибку во втором бите
code_with_error = list(code)
code_with_error[1] = 1 - code_with_error[1]
print("Случайная ошибка:", code_with_error)

# Декодируем и исправляем
fixed_data = decode_hamming(code_with_error)
print("Восстановленные данные:", fixed_data)
```

Список источников I